



МИНОБРНАУКИ РОССИИ

**федеральное государственное автономное образовательное учреждение
высшего образования**

**«Московский государственный технологический университет
«СТАНКИН»**

(ФГАОУ ВО «МГТУ «СТАНКИН»)

Институт
информационных
технологий

Кафедра
информационных систем

**Основная образовательная программа 09.03.02
«Информационные системы и технологии»**

Отчет по дисциплине «Структуры и алгоритмы обработки данных»

Тема: «Лабораторная работа №5»

**Проверил
К.т.н., доцент**

Ковалев И.А.

подпись

**Выполнил
студент группы ИДБ-25-06**

Грачев А.А.

подпись

Москва, 2026 г.

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ	3
1. РЕАЛИЗАЦИЯ БИНАРНОГО ДЕРЕВА ПОИСКА (BST) И ОБХОДОВ	4
2. 104. MAXIMUM DEPTH OF BINARY TREE	7
3. 98. VALIDATE BINARY SEARCH TREE	8
4. 230. KTH SMALLEST ELEMENT IN A BST	9
5. 236. LOWEST COMMON ANCESTOR OF A BINARY TREE	10
6. 124. BINARY TREE MAXIMUM PATH SUM	11
ЗАКЛЮЧЕНИЕ.....	12

ВВЕДЕНИЕ

Целью лабораторной работы является освоение рекурсивных алгоритмов на древовидных структурах данных, изучение бинарного дерева поиска (BST) и реализация его основных операций, а также применение четырёх классических способов обхода дерева. В практической части реализован BST с вставкой, поиском и удалением, написаны все четыре обхода (inorder, preorder, postorder, level-order), а также решены пять задач с платформы LeetCode, покрывающих ключевые паттерны: рекурсивное вычисление глубины, проверку свойств BST, inorder-обход с подсчётом, поиск наименьшего общего предка и нахождение максимальной суммы пути.

1. РЕАЛИЗАЦИЯ БИНАРНОГО ДЕРЕВА ПОИСКА (BST) И ОБХОДОВ

Класс `TreeNode` представляет узел дерева. Класс `BST` содержит ссылку на корень и реализует методы `insert`, `search`, `delete`, а также четыре обхода. Удаление реализовано с поиском `inorder`-преемника.

Код:

```
class TreeNode:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None

class BST:
    def __init__(self):
        self.root = None

    def insert(self, key):
        def _insert(node, key):
            if not node:
                return TreeNode(key)
            if key < node.key:
                node.left = _insert(node.left, key)
            elif key > node.key:
                node.right = _insert(node.right, key)
            return node
        self.root = _insert(self.root, key)

    def search(self, key):
        def _search(node, key):
            if not node:
                return False
            if key == node.key:
                return True
            elif key < node.key:
                return _search(node.left, key)
            else:
                return _search(node.right, key)
        return _search(self.root, key)

    def delete(self, key):
        def _min_value_node(node):
            while node.left:
                node = node.left
            return node

        def _delete(node, key):
            if not node:
                return None
            if key < node.key:
                node.left = _delete(node.left, key)
            elif key > node.key:
                node.right = _delete(node.right, key)
            else:
                if not node.left:
                    return node.right
                if not node.right:
                    return node.left
                min_node = _min_value_node(node.right)
                node.key = min_node.key
                min_node.right = _delete(min_node, min_node.key)
            return node
```

```

        if not node:
            return node
        if key < node.key:
            node.left = _delete(node.left, key)
        elif key > node.key:
            node.right = _delete(node.right, key)
        else:
            # один или ноль потомков
            if not node.left:
                return node.right
            elif not node.right:
                return node.left
            # два потомка
            temp = _min_value_node(node.right)
            node.key = temp.key
            node.right = _delete(node.right, temp.key)
        return node
    self.root = _delete(self.root, key)

# Обходы
def inorder(self):
    res = []
    def _in(node):
        if node:
            _in(node.left)
            res.append(node.key)
            _in(node.right)
    _in(self.root)
    return res

def preorder(self):
    res = []
    def _pre(node):
        if node:
            res.append(node.key)
            _pre(node.left)
            _pre(node.right)
    _pre(self.root)
    return res

def postorder(self):
    res = []
    def _post(node):
        if node:
            _post(node.left)
            _post(node.right)
            res.append(node.key)
    _post(self.root)
    return res

```

```

def level_order(self):
    if not self.root:
        return []
    from collections import deque
    q = deque([self.root])
    res = []
    while q:
        node = q.popleft()
        res.append(node.key)
        if node.left:
            q.append(node.left)
        if node.right:
            q.append(node.right)
    return res

```

Тесты:

```

bst = BST()
for val in [5, 3, 7, 2, 4, 6, 8]:
    bst.insert(val)
print("Inorder:", bst.inorder())          # [2, 3, 4, 5, 6,
7, 8]
print("Preorder:", bst.preorder())        # [5, 3, 2, 4, 7,
6, 8]
print("Postorder:", bst.postorder())      # [2, 4, 3, 6, 8,
7, 5]
print("Level-order:", bst.level_order()) # [5, 3, 7, 2, 4,
6, 8]
print("Search 4:", bst.search(4))         # True
bst.delete(5)
print("After deleting 5, inorder:", bst.inorder()) # [2,
3, 4, 6, 7, 8]

```

2. 104. MAXIMUM DEPTH OF BINARY TREE

Описание: Найти максимальную глубину бинарного дерева (количество узлов вдоль самого длинного пути от корня до листа).

Решение: Рекурсивно вычисляем глубину левого и правого поддеревьев, возвращаем $1 + \max(\text{left}, \text{right})$. Базовый случай — пустой узел (0). Время $O(n)$, память $O(h)$ для стека рекурсии.

Код:

```
from typing import Optional

class Solution:
    def maxDepth(self, root: Optional[TreeNode]) -> int:
        if not root:
            return 0
        return 1 + max(self.maxDepth(root.left),
self.maxDepth(root.right))
```

Тесты:

```
# Дерево [3, 9, 20, null, null, 15, 7]
root = TreeNode(3)
root.left = TreeNode(9)
root.right = TreeNode(20, TreeNode(15), TreeNode(7))
sol = Solution()
print(sol.maxDepth(root))    # 3
```

3. 98. VALIDATE BINARY SEARCH TREE

Описание: Проверить, является ли данное бинарное дерево корректным BST.

Решение: Рекурсивно передаём допустимый диапазон (low, high). Для каждого узла проверяем, что $\text{low} < \text{node.val} < \text{high}$, и рекурсивно проверяем левое поддерево с обновлённым high, правое — с обновлённым low. Базовый случай: пустой узел — True. Время $O(n)$, память $O(h)$.

Код:

```
class Solution:
    def isValidBST(self, root: Optional[TreeNode]) -> bool:
        def validate(node, low, high):
            if not node:
                return True
            if not (low < node.val < high):
                return False
            return (validate(node.left, low, node.val) and
                    validate(node.right, node.val, high))
        return validate(root, float('-inf'), float('inf'))
```

Тесты:

```
# Дерево [2,1,3] — валидное
root = TreeNode(2, TreeNode(1), TreeNode(3))
sol = Solution()
print(sol.isValidBST(root))    # True

# Дерево [5,1,4,null,null,3,6] — невалидное
root2 = TreeNode(5, TreeNode(1), TreeNode(4, TreeNode(3),
TreeNode(6)))
print(sol.isValidBST(root2))    # False
```


4. 230. KTH SMALLEST ELEMENT IN A BST

Описание: Найти k -й наименьший элемент в BST (нумерация с 1).

Решение: Выполняем inorder-обход (рекурсивно) и ведём счётчик посещённых узлов. Как только счётчик достигает k , сохраняем значение и прерываем обход. Время $O(n)$ в худшем, $O(k)$ в среднем, память $O(h)$.

Код:

```
class Solution:
    def kthSmallest(self, root: Optional[TreeNode], k: int)
-> int:
        self.count = 0
        self.result = None

        def inorder(node):
            if not node or self.result is not None:
                return
            inorder(node.left)
            self.count += 1
            if self.count == k:
                self.result = node.val
                return
            inorder(node.right)

        inorder(root)
        return self.result
```

Тесты:

```
# Дерево [3,1,4,null,2], k = 1
root = TreeNode(3, TreeNode(1, None, TreeNode(2)),
TreeNode(4))
sol = Solution()
print(sol.kthSmallest(root, 1))    # 1
```

5. 236. LOWEST COMMON ANCESTOR OF A BINARY TREE

Описание: Найти наименьшего общего предка (LCA) двух узлов в бинарном дереве (не обязательно BST).

Решение: Рекурсивный поиск снизу вверх. Если узел совпадает с одним из искомых, возвращаем его. Рекурсивно вызываем для левого и правого поддеревьев. Если оба вернули не None, текущий узел — LCA. Иначе возвращаем тот из результатов, который не None. Время $O(n)$, память $O(h)$.

Код:

```
class Solution:
    def lowestCommonAncestor(self, root: 'TreeNode', p:
'TreeNode', q: 'TreeNode') -> 'TreeNode':
        if not root or root == p or root == q:
            return root
        left = self.lowestCommonAncestor(root.left, p, q)
        right = self.lowestCommonAncestor(root.right, p, q)
        if left and right:
            return root
        return left if left else right
```

Тесты:

```
# Дерево [3,5,1,6,2,0,8,null,null,7,4], p=5, q=1
root = TreeNode(3)
root.left = TreeNode(5, TreeNode(6), TreeNode(2,
TreeNode(7), TreeNode(4)))
root.right = TreeNode(1, TreeNode(0), TreeNode(8))
p = root.left          # 5
q = root.right         # 1
sol = Solution()
print(sol.lowestCommonAncestor(root, p, q).val)  # 3
```

6. 124. BINARY TREE MAXIMUM PATH SUM

Описание: Найти максимальную сумму значений узлов вдоль любого пути в бинарном дереве (путь может начинаться и заканчиваться в любых узлах, но должен идти вниз по дереву и не может разветвляться).

Решение: Используем DFS, который вычисляет для каждого узла максимальный «вклад» в путь, проходящий через этот узел и уходящий вниз (не более чем по одной ветке). Параллельно обновляем глобальный максимум, включающий возможное разветвление (узел + левая ветка + правая ветка). Вклад возвращается как `node.val + max(left_gain, right_gain, 0)`.
Время $O(n)$, память $O(h)$.

Код:

```
class Solution:
    def maxPathSum(self, root: Optional[TreeNode]) ->
int:
        self.max_sum = float('-inf')

        def max_gain(node):
            if not node:
                return 0
            left = max(max_gain(node.left), 0)
            right = max(max_gain(node.right), 0)
            # путь с разветвлением через текущий узел
            self.max_sum = max(self.max_sum, node.val +
left + right)
            # возвращаем максимальный вклад, который
можно продлить вверх
            return node.val + max(left, right)

        max_gain(root)
        return self.max_sum
```

Тесты:

```
# Дерево [1,2,3]
root = TreeNode(1, TreeNode(2), TreeNode(3))
sol = Solution()
print(sol.maxPathSum(root))    # 6    (путь 2->1->3)
```

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы освоен главный паттерн решения задач на деревьях — рекурсивная обработка с базовым случаем и комбинированием результатов левого и правого поддеревьев. Реализованы все ключевые операции бинарного дерева поиска, включая корректное удаление узла с двумя потомками, и четыре способа обхода. На задачах LeetCode отработаны практические приёмы: вычисление высоты дерева (104), валидация BST с помощью диапазонов (98), использование inorder-обхода для нахождения k-го элемента (230), рекурсивный поиск LCA (236) и вычисление максимальной суммы пути с глобальным аккумулятором (124). Все алгоритмы протестированы на примерах с платформы и соответствуют заявленным временным и пространственным ограничениям.